Inatel

Sistemas Embarcados

Trabalho PID

Alunos:
Gustavo Pivoto Ambrósio - GEC - 2028
Marcelo Fróes Padilha Filho - GEC - 2071
Miguel Vianna Streva - GEC - 2011

Parte Teórica

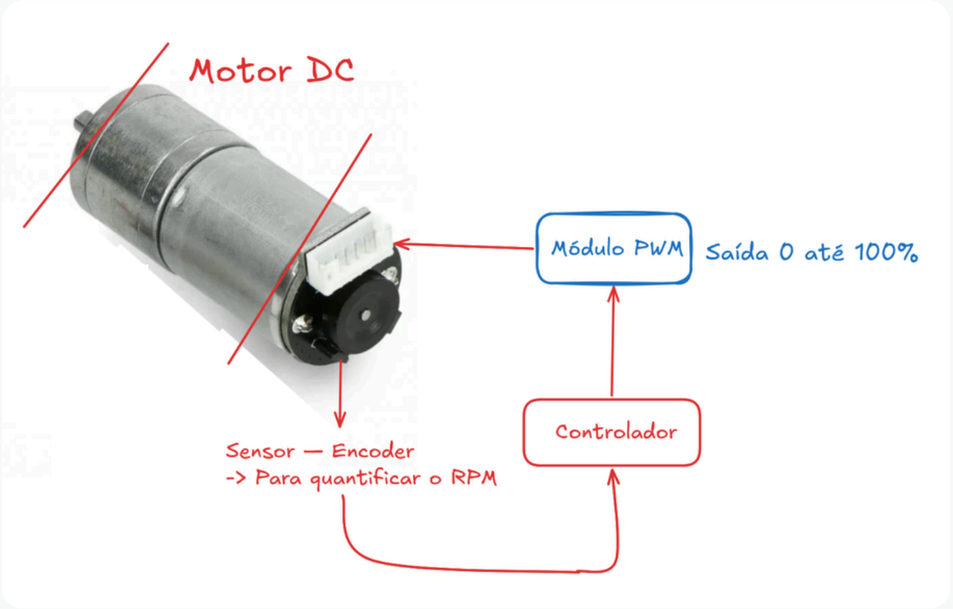
Funcionamento da planta

A planta consiste em um motor CC, cuja velocidade é controlada por um controlador PID através de um sinal PWM.

Principais componentes:

- Sensor: encoder incremental para medir a velocidade (RPM);
- Atuador: motor CC com driver de potência;
- Controlador: STM32 ou ESP32 para executar o PID e gerar o PWM.

O sistema opera em malha fechada, ajustando continuamente o PWM para manter a velocidade do motor próxima ao SetPoint desejado.



Variáveis do processo e faixas de operação

Principais variáveis:

- Variável controlada: velocidade do motor (RPM);
- Variável manipulada: duty cycle do PWM;
- SetPoint: velocidade desejada pelo usuário;
- Perturbações: variações de carga, tensão, atrito e temperatura.

Grandeza	Faixa de operação
Velocidade do motor	0 até a velocidade máxima nominal do motor
PWM	0% a 100%
SetPoint	0 até a velocidade máxima operacional

Parte Prática

Identificação do modelo

Análise do dataset - Motor CC

```python
import scipy.io as sio
import numpy as np

d = sio.loadmat('Motor_G6.mat')

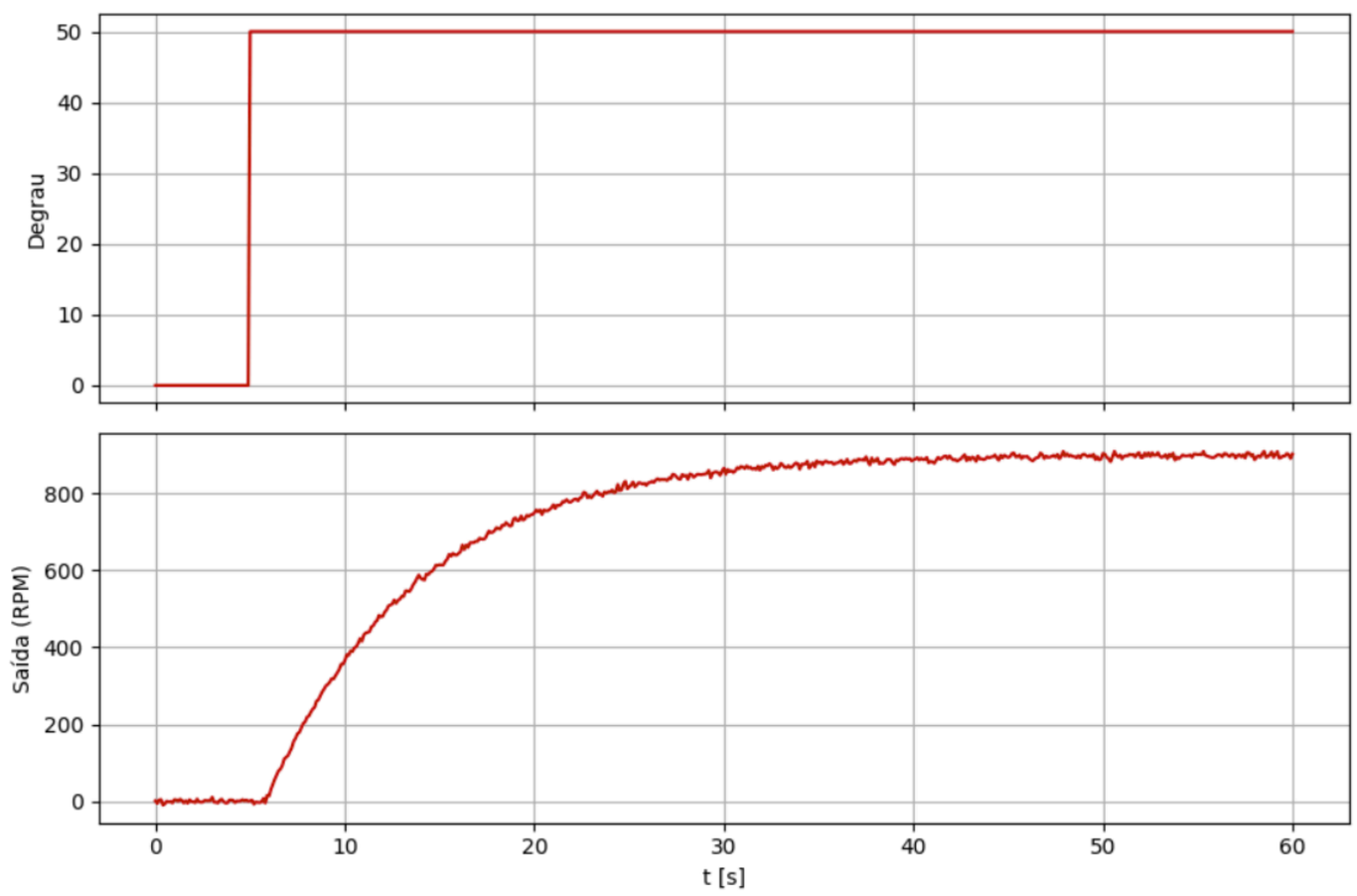
t = d['t'].ravel()
u = d['Degrau'].ravel()
y = d['Saida'].ravel()

print('Tamanho:', len(t))
print('t: de', t[0], 'até', t[-1])
print('Degrau: de', u[0], 'até', u[-1])
print('Saída (RPM) início:', y[:5])
print('Saída (RPM) fim:', y[-5:])
```

```
Tamanho: 601
t: de 0.0 até 60.0
Degrau: de 0.0 até 50.0
Saída (RPM) início: [ 1.5235854  -5.19992053  3.75225598  4.70282358 -9.75517594]
Saída (RPM) fim: [896.31658941 902.99595971 900.15450694 890.08448845 901.54927856]
```

Identificação do modelo

Análise do dataset - Motor CC



- *Escolha do método Sundaresan*

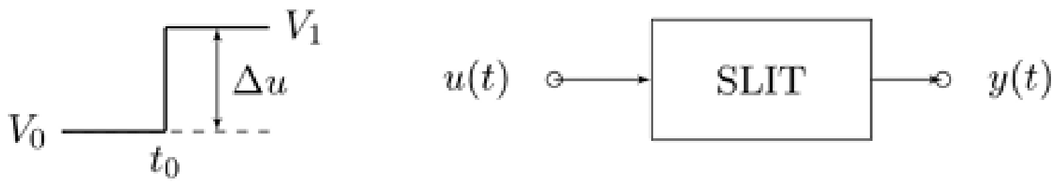
Identificação do modelo

Método Sundaresan

```
1 idx_step = np.where(np.diff(u) != 0)[0][0]
2 t0_step = t[idx_step + 1]
3 print('Índice do degrau:', idx_step)
4 print('Instante do degrau (t0):', t0_step)
```

```
Índice do degrau: 49
Instante do degrau (t0): 5.0
```

Figura 7: Modelo de Sistema para Ensaio da Curva de Reação.



- As curvas-modelo assumem implicitamente que o degrau (o t0 da Figura 7) coincide com a origem do tempo.
- No nosso ensaio real, esse t0 não é zero, é 5 segundos

Identificação do modelo

Cálculo do k (FOPDT)

```
y0 = y[:idx_step+1].mean()   # média da saída ANTES do degrau (deveria ser ~0)
y1 = y[-20:].mean()          # média da saída no final (regime permanente)

dy = y1 - y0
du = u[-1] - u[0]
k = dy / du

print('y0 (antes do degrau):', y0)
print('y1 (regime permanente):', y1)
print('Δy:', dy, '  Δu:', du)
print('k =', k)
```

```
y0 (antes do degrau): 0.45605515246505673
y1 (regime permanente): 898.0365191216958
Δy: 897.5804639692308   Δu: 50.0
k = 17.951609279384616
```

- y0: média da saída antes do degrau (regime inicial) - usa idx_step+1, pra pegar só os pontos anteriores à mudança.
- y1: média da saída no final do ensaio (regime permanente, já estabilizado) - só pega os últimos 20 pontos.
- dy = y1 - y0: variação total da saída, enquanto du = u[-1] - u[0] é a variação total da entrada (os 50 do degrau).
- k = dy/du: o ganho estático do sistema: RPM ganhos por unidade de degrau aplicada.

Identificação do modelo

Método Sundaresan

```python
def t_at(frac):
    """Acha o instante em que y cruza frac*Δy acima de y0, por interpolação linear."""
    target = y0 + frac * dy
    idx = np.where(y >= target)[0][0]   # primeiro ponto que já passou do alvo
    tA, tB = t[idx-1], t[idx]
    yA, yB = y[idx-1], y[idx]
    return tA + (target - yA) / (yB - yA) * (tB - tA)

t1 = t_at(0.353) - t0_step   # referência relativa ao instante do degrau
t2 = t_at(0.853) - t0_step

print('t1 =', t1)
print('t2 =', t2)
```

```
t1 = 4.286531829657502
t2 = 15.9512098717811
```

Identificação do modelo

Método Sundaresan

```
1 tau = (2/3) * (t2 - t1)
2 theta = 1.3*t1 - 0.29*t2
3
4 print('k     =', k)
5 print('tau   =', tau)
6 print('theta =', theta)
```

```
k     = 17.951609279384616
tau   = 7.776452028082398
theta = 0.9466405157382338
```

Identificação do modelo

Método Sundaresan

```
1 tau = (2/3) * (t2 - t1)
2 theta = 1.3*t1 - 0.29*t2
3
4 print('k     =', k)
5 print('tau   =', tau)
6 print('theta =', theta)
```

```
k     = 17.951609279384616
tau   = 7.776452028082398
theta = 0.9466405157382338
```

Identificação do modelo

Aproximação de Padé

- É feita a aproximação de Padé, já que é necessário transofrmar o atraso puro em função racional.

$$H(s) = \frac{k}{\tau s + 1} \cdot e^{-\theta s} \approx \frac{k}{\tau s + 1} \cdot \frac{\text{num_pade}(s)}{\text{den_pade}(s)}$$

Identificação do modelo

Aproximação de Padé

```python
import control as cnt

H = cnt.tf([k], [tau, 1])          # planta de 1ª ordem: k / (tau*s + 1)

n_pade = 10                         # ordem da aproximação de Padé
num_pade, den_pade = cnt.pade(theta, n_pade)
H_atraso = cnt.tf(num_pade, den_pade)

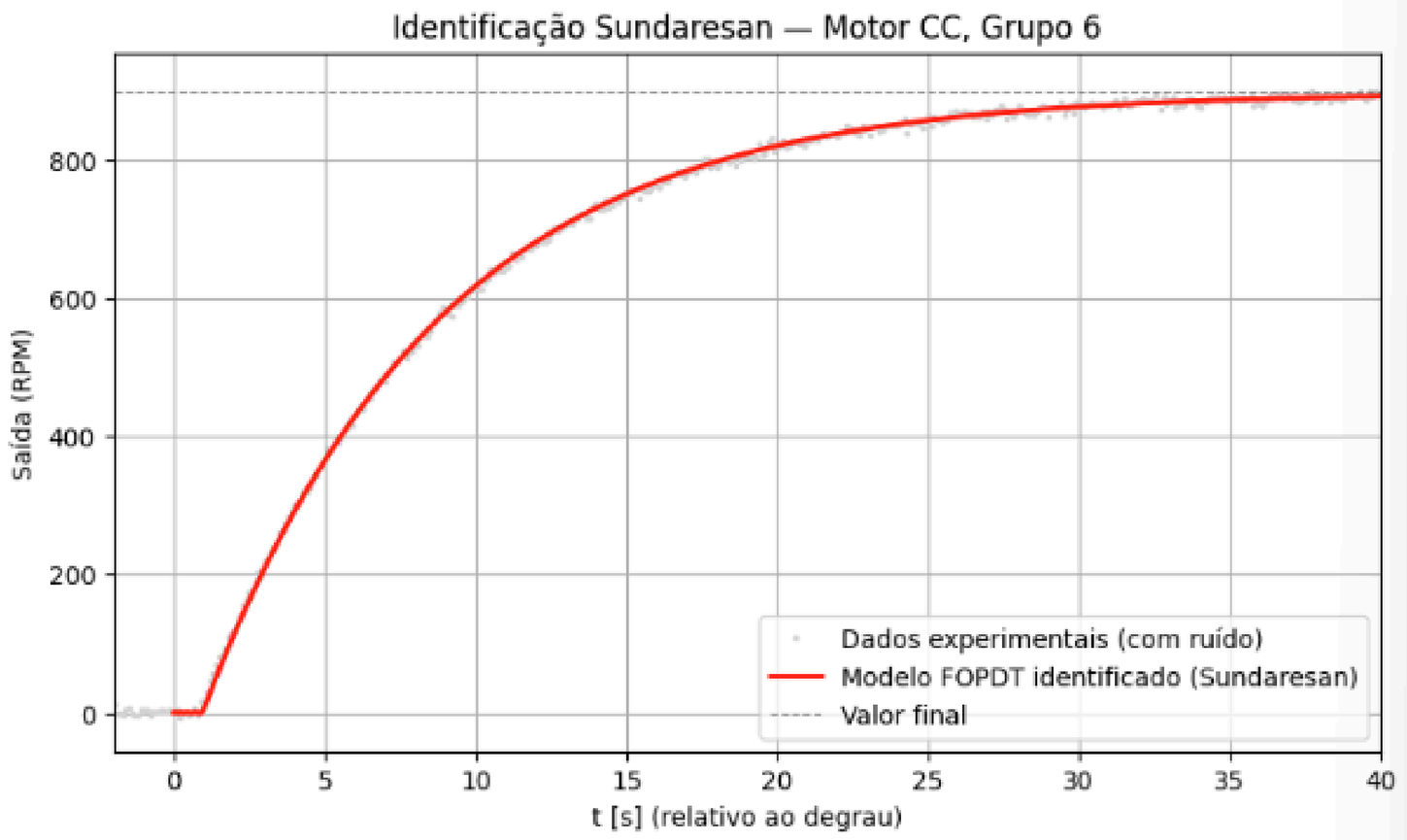
H_modelo = cnt.series(H, H_atraso)  # junta planta + atraso em série
print(H_modelo)
```

```
<TransferFunction>: sys[2]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

          17.95 s^10 - 2086 s^9 + 1.19e+05 s^8 - 4.358e+06 s^7 + 1.128e+08 s^6 - 2.144e+09 s^5 + 3.02e+10 s^4 - 3.1e+11 s^3 + 2.21e+12 s^2 - 9.858e+12 s + 2.083e+13
  ------------------------------------------------------------------------------------------------------------------------------------------------------------
7.776 s^11 + 904.6 s^10 + 5.166e+04 s^9 + 1.894e+06 s^8 + 4.91e+07 s^7 + 9.352e+08 s^6 + 1.32e+10 s^5 + 1.36e+11 s^4 + 9.747e+11 s^3 + 4.393e+12 s^2 + 9.571e+12 s + 1.16e+12
```

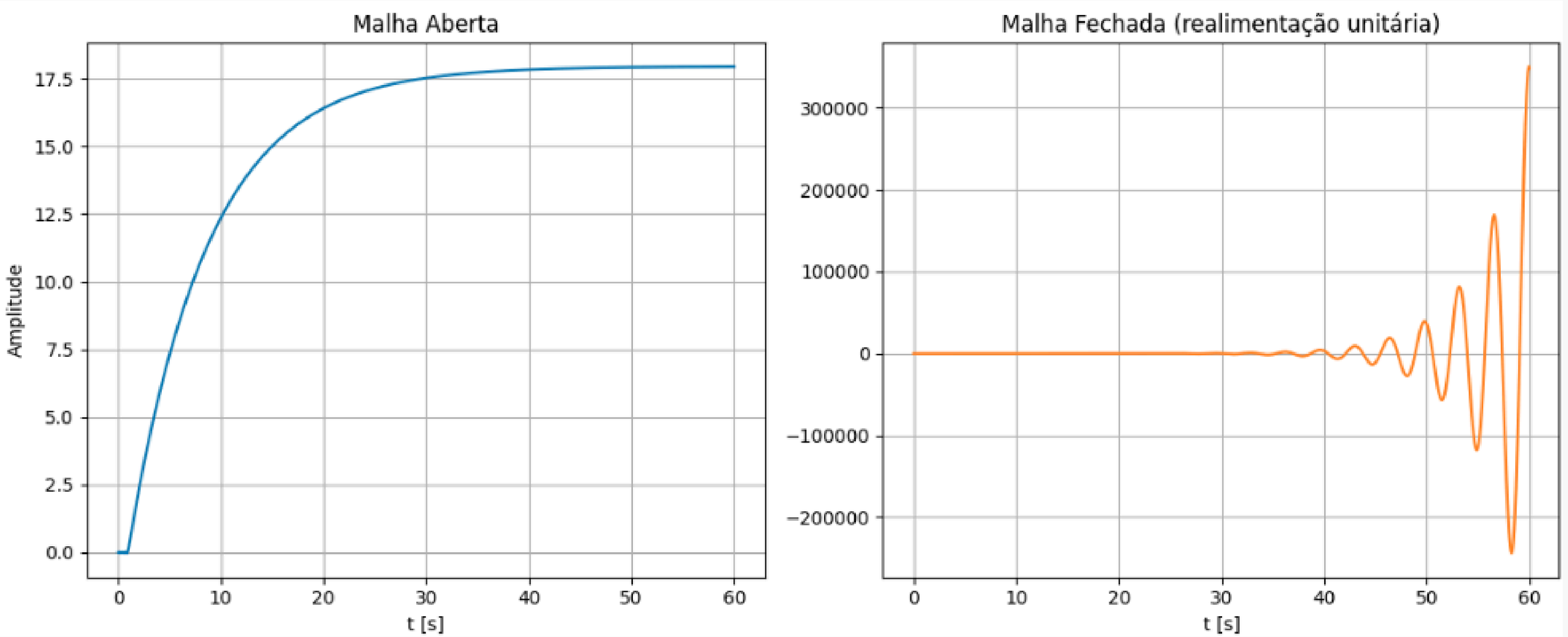
Identificação do modelo

Resposta Real x Modelo Estimado



Identificação do modelo

Malha Aberta x Malha Fechada



O modelo do motor (Sundaresan)

$G(s) = 17{,}95 \cdot e^{(-0{,}95 \cdot s)} / (7{,}78 \cdot s + 1)$

k · ganho	**τ · constante de tempo**	**θ · atraso**	**θ/τ · incontrolabilidade**
17,95 RPM por 1 % de PWM	**7,78 s quão lento ganha velocidade**	**0,95 s demora para começar a reagir**	**0,12 baixo: motor fácil de controlar**

- Sem controlador, a malha fechada é instável: ganho alto ($k \approx 18$) somado ao atraso faz a velocidade oscilar cada vez mais. Por isso precisamos sintonizar um PID.

Métodos de sintonia PID

TÉCNICA 3

CHR sem Sobrevalor

Resposta mais rápida possível sem passar do alvo.

$$Kp = 0{,}6 \cdot \tau / (k \cdot \theta)$$
$$Ti = \tau$$
$$Td = \theta / 2$$

TÉCNICA 6

ITAE

Minimiza o erro ponderado pelo tempo: resposta bem amortecida.

$$Kp = (A/k) \cdot (\theta/\tau)^{B}$$
$$Ti = \tau / (C + D \cdot \theta/\tau)$$
$$Td = \tau \cdot E \cdot (\theta/\tau)^{F} \quad A=0{,}965$$

Resultado da simulação (SP = 900 RPM)

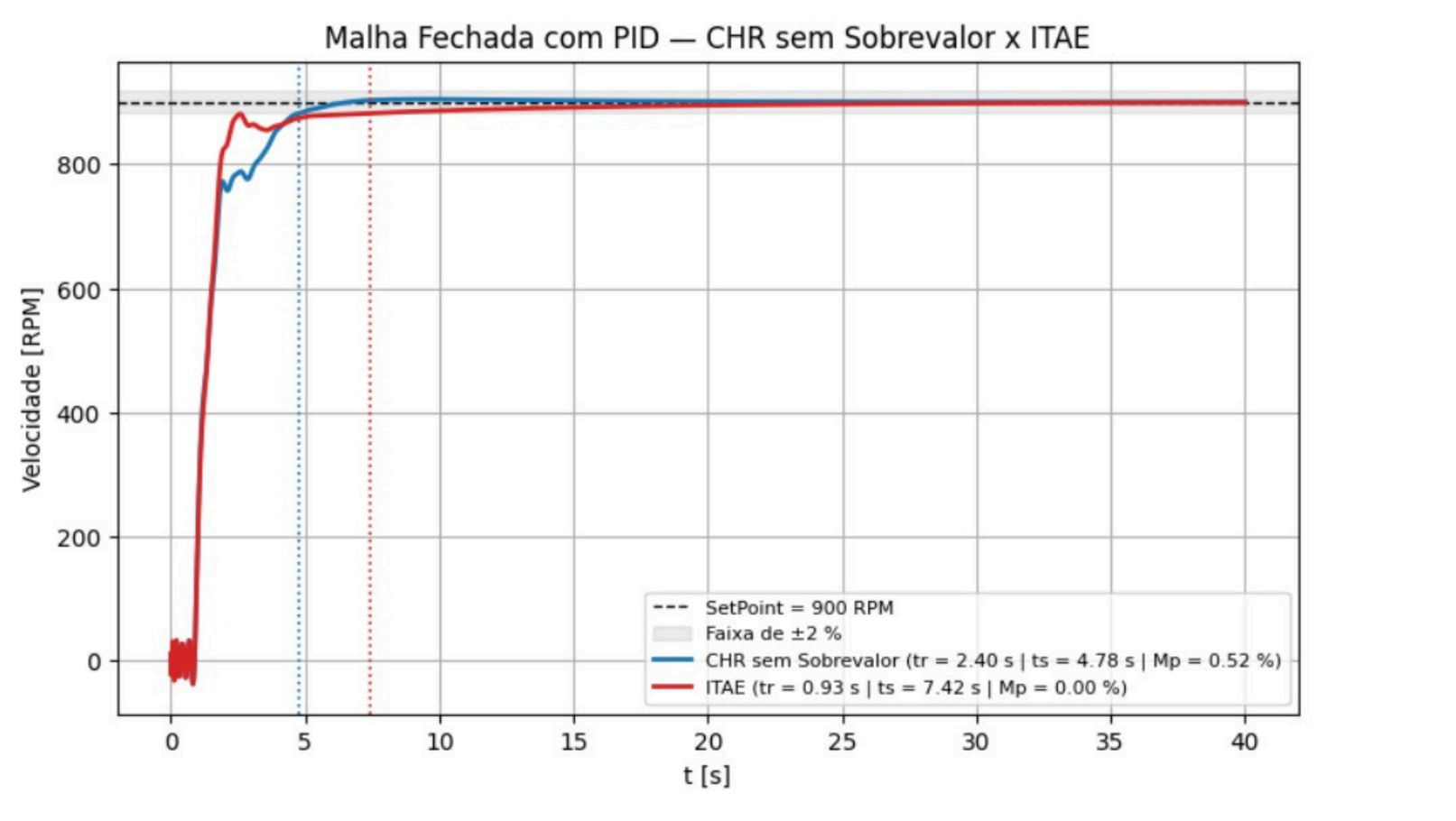
Método	Kp	Ti [s]	Td [s]	tr [s]	ts (2 %) [s]	Mp [%]	Erro em regime
CHR sem Sobrevalor	0,2746	7,7765	0,4733	2,40	**4,78**	0,52	0
ITAE	0,3220	9,9941	0,3386	**0,93**	7,42	0,00	0

ITAE tem Kp maior
0,322 × 0,275: reage mais forte no início.

CHR tem Ti = τ
7,78 s × 9,99 s do ITAE: integral mais forte.

CHR sem sobrevalor é o método mais adequado, sendo 36 % mais rápido que o **ITAE**.

Resposta em malha fechada



Azul · CHR Entra na faixa de ±2 % em 4,78 s.

Vermelho · ITAE Sobe antes, mas só acomoda em 7,42 s.

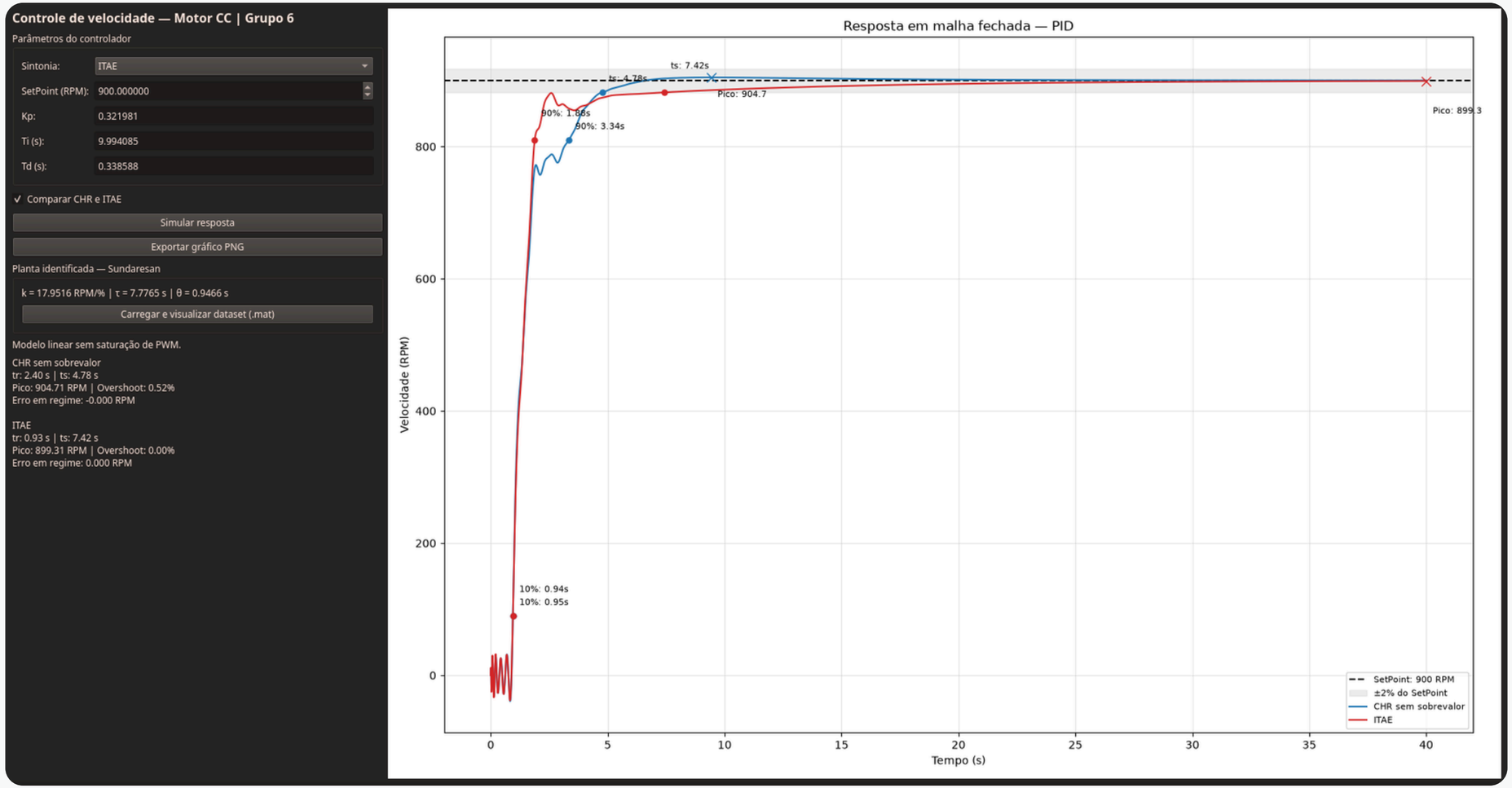
Perto de zero Oscilações no 1º segundo: artefato do Padé.

Interface

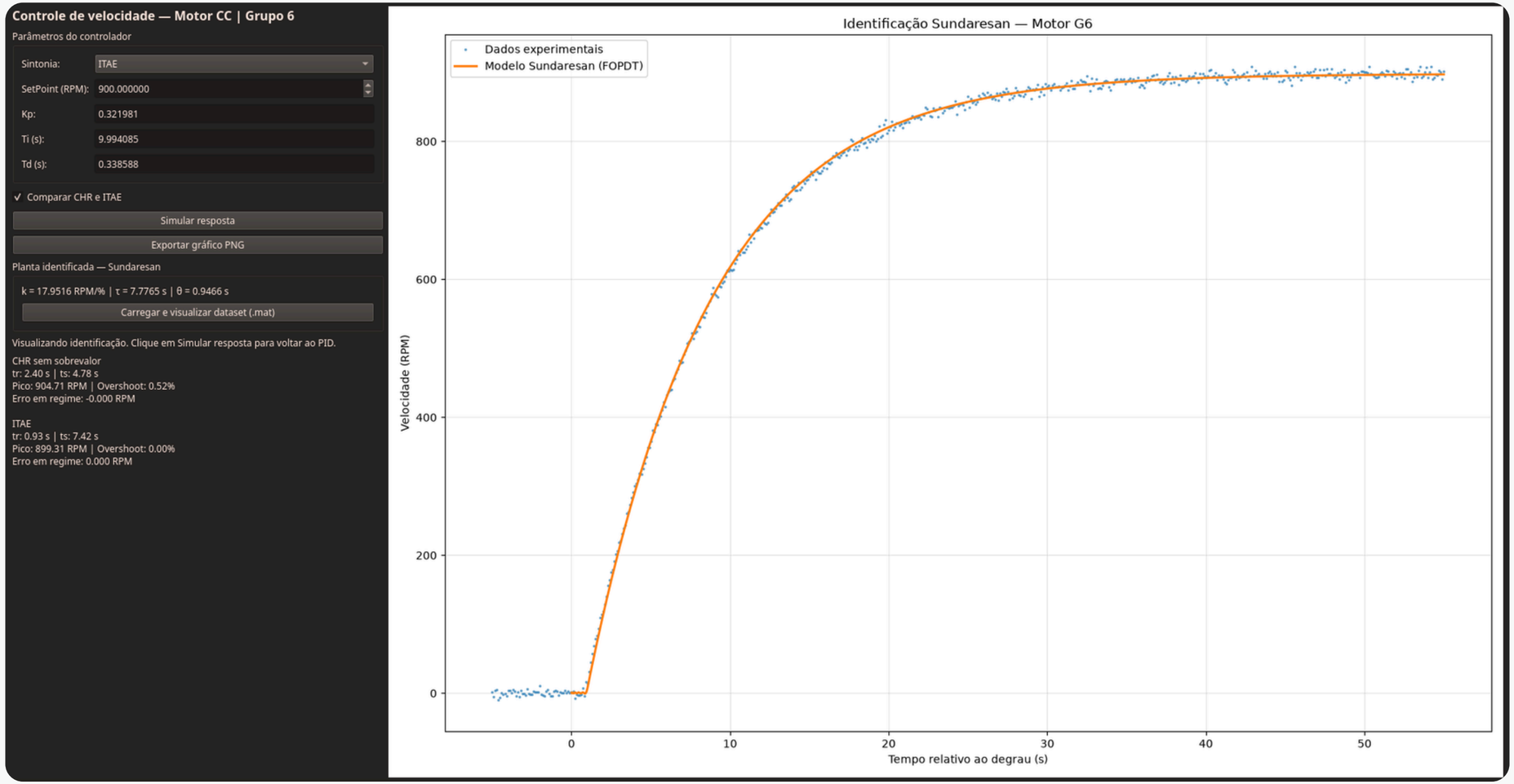
Foi desenvolvida uma interface em Python para facilitar a análise do sistema. A interface permite:

- selecionar o método de sintonia do controlador;
- ajustar manualmente os parâmetros do PID;
- visualizar a resposta do sistema em malha fechada;
- comparar os métodos CHR e ITAE;
- carregar o dataset experimental para análise da planta;
- exportar o gráfico gerado.

Interface


Controle de velocidade — Motor CC | Grupo 6
Parâmetros do controlador
Sintonia:
ITAE
SetPoint (RPM):
900.000000
Kp:
0.321981
Ti (s):
9.994085
Td (s):
0.338588
✓ Comparar CHR e ITAE
Simular resposta
Exportar gráfico PNG
Planta identificada — Sundaresan
k = 17.9516 RPM/% | τ = 7.7765 s | θ = 0.9466 s
Carregar e visualizar dataset (.mat)
Modelo linear sem saturação de PWM.
CHR sem sobrevalor
tr: 2.40 s | ts: 4.78 s
Pico: 904.71 RPM | Overshoot: 0.52%
Erro em regime: -0.000 RPM
ITAE
tr: 0.93 s | ts: 7.42 s
Pico: 899.31 RPM | Overshoot: 0.00%
Erro em regime: 0.000 RPM
Resposta em malha fechada — PID
ts: 7.42s
ts: 4.78s
Pico: 904.7
Pico: 899.3
90%: 1.88s
90%: 3.34s
10%: 0.94s
10%: 0.95s
Velocidade (RPM)
Tempo (s)
SetPoint: 900 RPM
±2% do SetPoint
CHR sem sobrevalor
ITAE

Interface


Controle de velocidade — Motor CC | Grupo 6
Parâmetros do controlador
Sintonia:
ITAE
SetPoint (RPM):
900.000000
Kp:
0.321981
Ti (s):
9.994085
Td (s):
0.338588
✓ Comparar CHR e ITAE
Simular resposta
Exportar gráfico PNG
Planta identificada — Sundaresan
k = 17.9516 RPM/% | τ = 7.7765 s | θ = 0.9466 s
Carregar e visualizar dataset (.mat)
Visualizando identificação. Clique em Simular resposta para voltar ao PID.
CHR sem sobrevalor
tr: 2.40 s | ts: 4.78 s
Pico: 904.71 RPM | Overshoot: 0.52%
Erro em regime: -0.000 RPM
ITAE
tr: 0.93 s | ts: 7.42 s
Pico: 899.31 RPM | Overshoot: 0.00%
Erro em regime: 0.000 RPM
Identificação Sundaresan — Motor G6
Dados experimentais
Modelo Sundaresan (FOPDT)
Velocidade (RPM)
0
200
400
600
800
0
10
20
30
40
50
Tempo relativo ao degrau (s)

Inatel

Dúvidas?

